

HPC Assignment — Parallel Bubble Sort and Parallel Merge Sort using OpenMP

This assignment implements:

Sequential and Parallel Bubble Sort
and

Sequential and Parallel Merge Sort
using:

- OpenMP
- odd-even transposition method
- recursion
- divide and conquer
- performance comparison

Code Reference: Uploaded Combined Bubble + Merge Sort Program

1. What We Have To Do?

We have to:

1. input array elements
2. sort array sequentially using Bubble Sort
3. sort array using Parallel Bubble Sort
4. sort array sequentially using Merge Sort
5. sort array using Parallel Merge Sort
6. compare execution times
7. display sorted arrays

Main Goal

To demonstrate:

how parallelism improves sorting performance
using OpenMP.

2. What is Bubble Sort?

Bubble Sort is a sorting algorithm where:
adjacent elements are repeatedly compared and swapped
until array becomes sorted.

Example

Array:

5 3 1 4

After sorting:

1 3 4 5

Why Called Bubble Sort?

Because:

- larger elements “bubble” toward end
- after every pass largest element reaches correct position

3. What is Merge Sort?

Merge Sort is:

divide-and-conquer sorting algorithm

It:

1. divides array into smaller halves
2. sorts each half recursively
3. merges sorted halves together

Example

Array:

5 3 1 4

Step 1:
 [5 3] [1 4]
 Step 2:
 [5] [3] [1] [4]
 Step 3:
 Merge:
 [3 5] [1 4]
 Final Merge:
 [1 3 4 5]

4. What is OpenMP?

OpenMP =
 Open Multi-Processing
 Used for:

- parallel programming in C/C++
- multicore CPU utilization
- thread-based execution

Important OpenMP Directives

#pragma omp parallel for

Converts loop into:

parallel loop

Different threads execute different loop iterations simultaneously.

#pragma omp parallel sections

Used for:

executing different code sections simultaneously.

omp_get_wtime()

Used for:

execution time measurement.

5. Overall Workflow of Program

Input Array

↓

Create Copies of Array

↓

Sequential Bubble Sort

↓

Parallel Bubble Sort

↓

Display Bubble Sort Results

↓

Sequential Merge Sort

↓

Parallel Merge Sort

↓

Display Merge Sort Results

↓

Compare Execution Times

6. Header Files

#include <iostream>

#include <vector>

#include <omp.h>

Purpose

Header	Purpose
iostream	input/output
vector	dynamic arrays

Namespace

using namespace std;

Avoids writing:

std::

again and again.

===== BUBBLE SORT =====

7. Sequential Bubble Sort

Function

void sequentialBubbleSort(vector<int> &arr)

Purpose:

performs normal sequential bubble sort.

Get Array Size

int n = arr.size();

Stores:

number of elements in array.

Outer Loop

for (int i = 0; i < n - 1; i++)

Controls:

number of passes.

Need:

n-1 passes

because after every pass:

one element reaches correct position.

Inner Loop

for (int j = 0; j < n - i - 1; j++)

Compares adjacent elements.

Why n-i-1 ?

After every pass:

- largest element already fixed at end
- no need to compare again

Swap Condition

if (arr[j] > arr[j + 1])

Checks:

wrong order.

Swap Operation

swap(arr[j], arr[j + 1]);

Exchanges elements.

Example

Before:

5 3

After Swap:

3 5

8. Parallel Bubble Sort

MOST IMPORTANT PART

Function:

void parallelBubbleSort(vector<int> &arr)

Uses:

Odd-Even Transposition Sort.

Problem with Normal Bubble Sort Parallelization

Normal bubble sort cannot be directly parallelized because:
adjacent swaps may overlap

Example

Array:

5 4 3

Comparisons:

(5,4)

(4,3)

If both happen simultaneously:

- conflicts occur
- incorrect swapping possible

Solution → Odd-Even Transposition Sort

Parallel version uses:

even phase

and

odd phase

9. Even Phase

Code:

```
#pragma omp parallel for  
for (int j = 0; j < n - 1; j += 2)
```

What Happens?

Compare pairs:

(0,1)

(2,3)

(4,5)

These pairs:

DO NOT overlap

So:

safe parallel execution

Example

Array:

5 3 8 2

Parallel Comparisons:

- (5,3)
- (8,2)

After Swaps:

3 5 2 8

Why #pragma omp parallel for Used?

Converts loop into:

parallel execution

Different threads process different pairs simultaneously.

Example

Thread 1:

compare (0,1)

Thread 2:

compare (2,3)

Both execute:

at same time.

10. Odd Phase

Code:

```
#pragma omp parallel for  
for (int j = 1; j < n - 1; j += 2)
```

What Happens?

Compare:

(1,2)

(3,4)

Again:

non-overlapping pairs

Example

From:

3 5 2 8

Compare:

(5,2)

Swap:

3 2 5 8

Why Odd-Even Method Works?

Because:

- simultaneous swaps never touch same elements
- avoids race conditions
- ensures thread safety

How Parallelism Happens in Bubble Sort?

Suppose:

8 elements

Different threads compare:

- different adjacent pairs
- simultaneously

This improves:

- CPU utilization
- execution speed

===== **MERGE SORT** =====

11. Merge Function

Function

```
void merge(vector<int>& arr, int left, int mid, int right)
```

Purpose:

merges two sorted halves.

Example

Left Half:

1 4 7

Right Half:

2 5 6

Merged Result:

1 2 4 5 6 7

Size Calculation

```
int n1 = mid - left + 1;
```

```
int n2 = right - mid;
```

Stores:

size of left and right subarrays.

Temporary Arrays

```
vector<int> L(n1), R(n2);
```

Purpose:
store left and right halves temporarily.

Copy Left Half

`L[i] = arr[left + i];`
Copies:
left subarray elements.

Copy Right Half

`R[j] = arr[mid + 1 + j];`
Copies:
right subarray elements.

Merge Logic

Code:
`while (i < n1 && j < n2)`
Compares:
left and right arrays.

Comparison

`if (L[i] <= R[j])`
Smaller element inserted first.

Remaining Elements

Extra loops:
`while (i < n1)`
`while (j < n2)`
copy remaining elements.

12. Sequential Merge Sort

Function

`void sequentialMergeSort(vector<int>& arr, int left, int right)`
Uses:
recursive divide-and-conquer approach.

Base Condition

`if (left >= right)`
 `return;`
Stops recursion when:
only one element remains.
Single element:
already sorted.

Find Middle

`int mid = left + (right - left) / 2;`
Divides array into halves.

Recursive Calls

`sequentialMergeSort(arr, left, mid);`
`sequentialMergeSort(arr, mid + 1, right);`
Sorts:

- left half
- right half

recursively.

Merge Sorted Halves

`merge(arr, left, mid, right);`

Combines sorted halves.

13. Parallel Merge Sort

MOST IMPORTANT PART

Function:

```
void parallelMergeSort(vector<int>& arr, int left, int right, int depth)
```

Uses:

OpenMP parallel sections.

Why Merge Sort Suitable for Parallelism?

Because:

left half and right half are independent

Both can be sorted simultaneously.

Parallel Sections

Code:

```
#pragma omp parallel sections
```

Creates:

parallel execution blocks.

Section 1

```
#pragma omp section
```

```
parallelMergeSort(arr, left, mid, depth - 1);
```

Sorts:

left half.

Section 2

```
#pragma omp section
```

```
parallelMergeSort(arr, mid + 1, right, depth - 1);
```

Sorts:

right half.

Both Sections Execute Simultaneously

Example:

Thread	Work
T1	left half
T2	right half

Why Depth Parameter Used?

Code:

```
if (depth <= 0)
```

Purpose:

limits excessive thread creation.

Without limit:

- too many threads created
- huge overhead occurs
- performance becomes slower

Important Reality

For small arrays:

parallel merge sort may appear slower because:

- thread creation overhead
- synchronization cost
- recursion overhead

Parallelism mainly helps for:

large datasets.

14. Main Function

int main()

Program execution starts here.

Input Number of Elements

int n;

Stores:

array size.

Create Array

vector<int> arr(n);

Dynamic array created.

Input Elements

for (int i = 0; i < n; i++)

Reads:

array elements.

Create Copies of Array

vector<int> bubbleSeq = arr;

vector<int> bubblePar = arr;

Purpose:

Both algorithms should sort:

same original data

for fair comparison.

Similarly:

vector<int> mergeSeq = arr;

vector<int> mergePar = arr;

15. Timing Sequential Bubble Sort

double start = omp_get_wtime();

Records:

starting time.

Run Sequential Bubble Sort

sequentialBubbleSort(bubbleSeq);

Executes sequential bubble sort.

End Time

double end = omp_get_wtime();

Stores ending time.

Execution Time

(end - start)

Calculates:

execution duration.

16. Timing Parallel Bubble Sort

Same process repeated for:

parallel bubble sort.

Code:

parallelBubbleSort(bubblePar);

17. Timing Sequential Merge Sort

Code:

sequentialMergeSort(mergeSeq, 0, n - 1);

Runs sequential merge sort.

18. Timing Parallel Merge Sort

Code:

```
parallelMergeSort(mergePar, 0, n - 1, 3);
```

Runs parallel merge sort.

Depth value:

3

Limits recursive parallelism.

Display Sorted Arrays

Loops print:

sorted arrays.

19. Example Execution

Input

5 3 4 1 2 6

Sequential Bubble Result

1 2 3 4 5 6

Parallel Bubble Result

1 2 3 4 5 6

Sequential Merge Result

1 2 3 4 5 6

Parallel Merge Result

1 2 3 4 5 6

20. Why Parallel Sorting Faster?

Because:

- multiple operations execute simultaneously
- multicore CPU utilized
- work divided among threads

Important Observation

For very small arrays:

parallel algorithms may appear slower because:

- thread creation overhead dominates
- synchronization overhead exists

For large arrays:

parallelism improves performance significantly.

21. Compilation Command

```
g++ -fopenmp sort.cpp -o sort
```

Run

```
./sort
```

Why -fopenmp Needed?

Enables:

OpenMP support.

Without it:

```
#pragma omp
```

ignored.

22. Time Complexity

Sequential Bubble Sort

$O(n^2)$

Parallel Bubble Sort

Approximate:

$O(n^2 / P)$

Where:

- P = processors/threads

Why Still $O(n^2)$?

Because:

- total comparisons still quadratic
- work only distributed among threads

Sequential Merge Sort

$O(n \log n)$

Parallel Merge Sort

Approximate:

$O((n \log n) / P)$

Where:

- P = processors/threads

23. Advantages of Parallel Sorting

Advantage	Explanation
Faster execution	multicore utilization
Efficient large datasets	work distributed among threads
Better CPU utilization	simultaneous execution
Improved performance	reduced practical execution time

24. Important Viva Questions

Q1. What is Bubble Sort?

Sorting algorithm using repeated adjacent swaps.

Q2. Why Bubble Sort difficult to parallelize directly?

Because adjacent swaps may overlap.

Q3. What is Odd-Even Transposition Sort?

Parallel variation of bubble sort using odd and even phases.

Q4. Why even and odd phases used?

To avoid overlapping swaps and race conditions.

Q5. What is Merge Sort?

Divide-and-conquer sorting algorithm.

Q6. Why Merge Sort suitable for parallelism?

Left and right halves are independent.

Q7. What does `#pragma omp parallel for` do?

Executes loop iterations in parallel.

Q8. What does `#pragma omp parallel sections` do?

Runs different sections simultaneously.

Q9. Why `omp_get_wtime()` used?

To measure execution time.

Q10. Why array copies used?

To compare algorithms fairly using same input.

Q11. Why depth parameter used?

To limit excessive thread creation.

Q12. What is recursion?

Function calling itself repeatedly.

Q13. What is divide and conquer?

Technique of dividing problem into smaller subproblems.

Q14. Why parallel sorting may appear slower sometimes?

Because thread overhead dominates for small inputs.

Q15. What is time complexity of Bubble Sort?

$O(n^2)$

Q16. What is time complexity of Merge Sort?

$O(n \log n)$

25. Short Practical Explanation

This program implements Sequential and Parallel Bubble Sort along with Sequential and Parallel Merge Sort using OpenMP. The parallel Bubble Sort uses Odd-Even Transposition method, where non-overlapping adjacent pairs are compared simultaneously using `#pragma omp parallel for`. Parallel Merge Sort uses recursive divide-and-conquer with `#pragma omp parallel` sections to sort left and right halves simultaneously. Execution times are measured using `omp_get_wtime()` to compare sequential and parallel performance. The assignment demonstrates multicore parallelism, recursion, sorting optimization, and thread-based execution using OpenMP.

From <<https://chatgpt.com/c/6a05a79c-df14-8321-87b7-d41b1c4bf962>>